DATA CONSISTENCY PATTERNS IN MICROSERVICE ARCHITECTURE V1

Distributed transactions, reliable messaging and the guarantees each pattern actually gives you

the dual-write problem · saga & its missing isolation · outbox · idempotency · event sourcing · CQRS · locks & fencing

1 · THE PROBLEM THESE PATTERNS EXIST TO SOLVE

start here — what breaks when one transaction becomes several services, and what you can still guarantee

What you had, and what you gave up

In a monolith with one database, "place an order" is one transaction. Deduct stock, take payment, create the order — **all of it commits, or none of it does**, and no other transaction sees the half-finished state.

Split those into three services with three databases and that guarantee is simply gone. There is no transaction spanning them. Something can succeed in one service and fail in the next, and there is no mechanism that automatically undoes the first.

The dual-write problem — the specific failure

Almost every pattern on this sheet exists because of this one thing: **you cannot atomically write to a database and send a message.**

```
// looks fine. is not.
db.save(order)          // committed ✓
broker.publish(event)   // process crashes here ✗
→ the order exists, nobody downstream knows

// reversing the order is no better
broker.publish(event)   // published ✓
db.save(order)          // rolls back ✗
→ downstream reacts to an order that never existed
```

There is no ordering of those two lines that is safe, and no amount of retry logic fixes it. That is the reason the outbox pattern exists — see panel 7.

What you can still have

Guarantee	Available?
ACID within one service	**Yes.** Keep data that changes together in one database and one transaction. This is the most under-used tool on the sheet
Atomicity across services	Only via blocking protocols nobody wants at scale — or approximated by compensation
Isolation across services	**No.** Intermediate states *are* visible. This is the guarantee people forget they lost
Eventual convergence	**Yes** — with reliable delivery and idempotent consumers
Exactly-once delivery	**No.** At-least-once delivery plus idempotent processing gives the same *effect*

The reframe that makes this tractable. Stop asking "how do I get a distributed transaction?" and start asking "what is the business behaviour when step 3 of 5 fails?" That question has an answer for every real process — usually a reversal, a retry, a hold, or a human — and it is the answer the patterns implement.

Before reaching for any of this: check the boundary. If two services are constantly involved in the same transaction, that is often a service-boundary defect, not a consistency problem. Merging them is frequently the correct fix and removes the need for every pattern here.

The order to reach for things

#	Option
1	**One local transaction.** Redraw the boundary so the change is local. Free, and always correct
2	**Optimistic concurrency** — a version column. Handles concurrent updates to one record; costs almost nothing
3	**Outbox + idempotent consumer.** The workhorse. Reliable eventual consistency between services
4	**Saga** when a multi-step process genuinely needs undo semantics
5	**TCC** when the domain supports reservation, and you need a hold before committing
6	**2PC** only inside one trusted, low-latency boundary, and rarely

Most teams start at 4 and skip 1 to 3. A saga is a distributed state machine with compensation paths, failure handling and its own operational burden. Reach for it when the process is genuinely long-running and reversible — not as the default answer to "we have two services".

Vocabulary used throughout

Compensation	A business-level undo — refund, release, cancel. Not a rollback; the original event still happened
Idempotent	Applying it twice has the same effect as once
Semantic lock	A status flag marking a record as in-flight and not yet final
Convergence window	How long the system is observably inconsistent. Should be a number

2 · THE PATTERN CATALOGUE

what each one guarantees, what it costs, and where it belongs

#	Pattern	How it works	Guarantee	Good for	Not good for	Main cost	Example
1	**Two-phase commit** 2PC · synchronous	Coordinator asks all participants to prepare; if all vote yes, it tells them to commit	Strong — atomic across participants	A few services in one trusted, low-latency boundary; legacy XA estates	Cloud, cross-region, high latency, many participants, anything long-running	**Blocks** — participants hold locks until the coordinator returns. If it dies mid-flight they stay locked	Transfer between two accounts in one data centre
2	**Three-phase commit** 3PC · of historical interest	Adds a pre-commit phase so participants never sit uncertain	Non-blocking *only* under fail-stop with no partitions	Understanding why atomic commit is hard	**Production.** See the correction in panel 3	Under a network partition it can produce **inconsistency** — a worse failure than blocking	Effectively none; it is not used commercially
3	**Saga** choreography or orchestration	A sequence of local transactions; each has a compensating action if a later step fails	Atomicity, Consistency, Durability — **no Isolation**	Long-running business processes with a natural reversal: order, booking, fulfilment	Steps that cannot be undone; processes needing isolation from concurrent activity	Intermediate states are visible; compensation logic and its own failure paths	Order → payment → stock → shipping, with refund and release on failure
4	**TCC** try-confirm-cancel	**Try** reserves resources, **Confirm** makes it final, **Cancel** releases. Two-phase, at the application level	Eventual, with an explicit hold in between	Domains with a real notion of reservation — seats, stock, capacity, credit limit	Domains where nothing can be reserved; simple CRUD	Every service must implement all three operations; reservations need expiry	Hold a seat for 10 minutes, then confirm on payment or release on timeout
5	**Transactional outbox**	Write the state change **and** the event to an outbox table in **one** local transaction; a relay publishes from the table	At-least-once publication, atomic with the state change	**Almost every service that publishes events.** The default answer to the dual-write problem	Nothing much — this is close to a baseline requirement	An outbox table to grow and prune, and a relay process to run	Order service commits the order and `OrderCreated` together
6	**Inbox** idempotent consumer	Record each processed message id in the same transaction as its effect; skip anything already seen	Effectively-once processing over at-least-once delivery	Every consumer whose work is not naturally idempotent	Genuinely fire-and-forget, or naturally idempotent, work	An inbox table with a retention policy sized to the redelivery window	Payment service charges once even if the event arrives three times
7	**Change data capture** CDC	Read the database's own replication log and publish changes	At-least-once, near real time	Getting events out of systems you cannot change; replacing outbox polling	Databases without a usable log; when you want *business* events rather than row changes	Events are row-shaped, not domain-shaped, and leak your schema to consumers	Debezium streaming MySQL binlog into Kafka
8	**Event sourcing**	Store the sequence of changes as the source of truth; current state is derived by replaying them	Complete history; state is a projection	Audit requirements, temporal queries, rebuilding read models, domains where *why* matters	Simple CRUD; teams new to it; anything needing arbitrary ad-hoc queries over current state	Snapshots, schema versioning and erasure obligations — see panel 10	Account balance derived from every credit and debit
9	**CQRS**	Separate the write model from one or more read models, updated asynchronously	Eventual between write and read side	Read-heavy systems, expensive queries, several different read shapes over one write model	Systems needing immediate read-your-writes; simple applications	Two models to keep in step, and a visible replication lag users may notice	Orders written normalised; dashboards read a denormalised projection
10	**Optimistic concurrency**	A version column checked on update; a mismatch means someone else changed it first	Prevents lost updates on a single record	**Nearly everything.** The cheapest correctness tool available, and usually the missing one	Cross-record or cross-service invariants	Callers must handle a conflict and retry	`UPDATE … WHERE id=? AND version=?`
11	**Distributed lock / lease**	Acquire a lock with an expiry before entering a critical section	**Not mutual exclusion on its own** — see panel 12	Reducing contention; single-runner scheduled jobs; best-effort coordination	Correctness-critical exclusion without fencing tokens; anything long-running	Expiry during a stall gives two holders at once, silently	One instance runs the nightly job

These compose, and the common production combination is a stack rather than a choice: outbox to publish reliably, inbox for idempotent consumption, optimistic concurrency on the records themselves, and a saga only where a multi-step process genuinely needs undo.

3 · ATOMIC COMMIT — 2PC, AND WHY 3PC IS NOT THE ANSWER

the strongest guarantee, the reason it is rare, and a correction worth making

Two-phase commit

```
PHASE 1 — prepare
coordinator → all: "can you commit?"
participants: write to log, take locks, reply YES/NO

PHASE 2 — commit or abort
all YES → coordinator → all: "commit"
any NO  → coordinator → all: "abort"
```

The failure that matters is the coordinator dying between the phases. Every participant has voted yes and is holding locks, and none of them may unilaterally decide. They **block** — holding those locks, blocking unrelated traffic — until the coordinator recovers. In a cloud or cross-region deployment that window is long enough to be an outage.

When 2PC is still reasonable

- A small number of participants inside **one** trusted, low-latency network
- Short transactions, so the locking window is small
- A highly available coordinator with durable state
- Existing XA infrastructure that already works

It is not a microservices pattern. It couples the availability of every participant together — the composite availability is the product of them all — and that is precisely what service independence was meant to avoid.

Three-phase commit — the correction

3PC is not a hardened 2PC for unreliable networks. The opposite is true. Its non-blocking property holds **only** under a synchronous network with reliable failure detection and **no partitions**. Introduce a partition — the thing an unreliable network produces — and 3PC can commit on one side while aborting on the other, leaving **inconsistent state** when the partition heals.

Blocking is preferable to inconsistency. A blocked transaction resolves when the coordinator recovers. Divergent state after a partition may be unrecoverable, because there is no longer a record of which side was right. That trade is why **3PC has never been implemented in a commercial database system**, and why it appears in textbooks rather than in production.

What is actually used instead

Need	Use
Atomicity across services	**Saga** with compensation — accept the loss of isolation
Atomicity with a hold	**TCC** — reserve, then confirm or cancel
Agreement in a distributed store	**Consensus** — Raft or Paxos, which is partition-safe by design and is what distributed SQL uses
Reliable propagation	**Outbox + idempotent consumer**

Note the distinction. Consensus and atomic commit are different problems. Consensus needs a majority to agree on a value and tolerates a minority failing; atomic commit needs *unanimity* to commit, so a single unreachable participant blocks it. That is why Raft scales to production and 2PC does not.

4 · TCC

reserve, then decide

Every participating service implements three operations. It is a two-phase protocol like 2PC, but expressed in **business terms** at the application level — so nothing holds a database lock between the phases.

Operation	Does
Try	Reserve the resource. Stock is moved to *held*, not deducted; the seat is blocked, not sold
Confirm	Make it final. Must be idempotent, and must not fail for business reasons — everything was checked in Try
Cancel	Release the reservation. Must be idempotent, and must tolerate a Try that never happened

The difference from a saga. A saga commits each step for real and undoes it afterwards — so a customer may see a charge and then a refund. TCC holds the resource without committing, so the intermediate state is a *reservation* rather than a completed action nobody wanted.

What it demands of the domain

- **Reservation must be meaningful** — you can hold a seat or stock; you cannot hold an email that was sent
- **Reservations must expire**, or a crashed orchestrator leaks capacity indefinitely
- **Confirm and Cancel must be idempotent** and safe to arrive out of order
- **Cancel must handle the empty case** — arriving for a Try that never landed

Where it is misapplied. On domains with no notion of a hold, teams implement Try as "do it" and Cancel as "undo it" — at which point it is a saga with three times the code and none of the benefit.

The expiry is the part to get right. Too short and legitimate slow payments fail; too long and inventory is unavailable to real buyers. Make it a stated business decision, and monitor how many reservations expire unconfirmed.

5 · SAGA — CHOREOGRAPHY vs ORCHESTRATION

the same pattern with two very different operational characters

Choreography — services react to events

```
Order     ──OrderCreated──▶  Payment
Payment   ──PaymentDone──▶   Inventory
Inventory ──StockReserved─▶  Shipping

// on failure, a compensating event flows back
Inventory ──OutOfStock──▶    Payment  (refund)
Payment   ──Refunded──▶      Order    (cancel)
```

	Character
Good	No central component; services stay loosely coupled; easy to add a listener
Bad	**No single place describes the process.** Understanding it means reading every service. Cyclic dependencies creep in, and testing the whole flow is hard
Use when	Few steps, stable process, teams comfortable with event-driven debugging

Orchestration — a coordinator drives

```
        ┌──────────────┐
        │ Orchestrator │  holds the state machine
        └──────────────┘
   1▼ cmd    2▼ cmd    3▼ cmd
 Payment   Inventory  Shipping
   ▲ reply   ▲ reply   ▲ reply

// on failure it issues compensations in reverse
```

	Character
Good	The process is explicit and in one place; state is queryable; timeouts, retries and compensation ordering are centralised
Bad	A component to build and run; risks becoming a place business logic accumulates
Use when	More than a few steps, or the process itself matters to the business

What both require

- **Every step needs a compensating action**, and it must be defined before the step ships — not discovered during an incident
- **Compensations must be idempotent** and safe to retry
- **The saga's own state must be durable.** If the coordinator crashes mid-flight, it must resume — this is why workflow engines are commonly used
- **Timeouts at every step**, with a defined action when one expires
- **Correlation id** threaded through every message so a flow can be traced

The question most designs skip: what if the compensation itself fails? The refund is rejected, or the release times out. Retry with backoff, then a dead-letter queue, then a human — with an alert and a runbook. A saga with no answer here will eventually strand real orders in an intermediate state, and nobody will notice until a customer calls.

Steps that cannot be compensated

An email sent, a physical item dispatched, an external payment captured — none of these truly reverse. Put these steps **last**, after everything reversible has succeeded, so the irreversible action only happens once the outcome is certain.

Order the steps by reversibility. Retryable steps first, then compensable ones, then a pivot step that commits the outcome, then irreversible actions. This ordering removes most compensation complexity before it is written.

Prefer orchestration once the process exceeds about three steps. The cost is one component; the benefit is that the business process exists somewhere you can read, test, monitor and change — rather than being an emergent property of who happens to be subscribed to what.

6 · THE MISSING "I"

sagas give ACD, not ACID

A saga provides Atomicity, Consistency and Durability — but no Isolation. Each local transaction commits immediately, so **every other transaction in the system can see the intermediate state**. This is the single most under-stated property of the pattern, and the source of its subtlest production bugs.

The anomalies you inherit

Anomaly	What happens
Dirty read	Another saga reads data that a later compensation will undo — and acts on it
Lost update	Two sagas update the same record; one silently overwrites the other
Fuzzy read	A saga reads the same record twice and gets different values mid-flight

Countermeasures

Technique	How it helps
Semantic lock	Mark the record with an in-flight status — `PENDING`, `APPROVAL_PENDING` — so others know it is not final and can wait or refuse
Commutative updates	Design operations so order does not matter. `credit` and `debit` commute; `set balance` does not. Eliminates lost updates
Pessimistic view	Reorder the saga so risky updates happen in steps that will not be compensated, removing the dirty-read window
Reread value	Re-read and verify the record is unchanged before writing — optimistic concurrency, applied within a saga
Version file	Record operations and apply them in a defined order, making out-of-order arrival safe
By value	Route by risk — use a saga for low-value requests and a stricter mechanism for high-value ones

The semantic lock is the one to reach for first. It is cheap, explicit in the domain model, and it makes the in-flight state visible to both other services and the user interface — which is usually where the confusion would otherwise surface.

7 · RELIABLE MESSAGING — OUTBOX, INBOX AND CDC

the answer to the dual-write problem, and the pattern most services actually need

Outbox — publishing reliably

```
BEGIN                       // one local transaction
  INSERT INTO orders  …
  INSERT INTO outbox (id, type, payload, created_at)
       VALUES (uuid, 'OrderCreated', {…}, now())
COMMIT                      // both, or neither ✓

// separately, a relay:
poll outbox → publish to broker → mark sent
// crash before marking = republish = duplicate
// which is exactly why consumers must be idempotent
```

Why this works. The event now commits atomically with the state change, because they are the same transaction in the same database. The unreliable step — publishing — moved *out* of the critical path and became retryable.

Operating it

- **Prune the outbox** on a schedule; an unpruned table becomes the slowest part of your write path
- **Index on unsent + created_at** so the relay's poll stays cheap as the table grows
- **Preserve order per aggregate** if consumers depend on it — publish by aggregate id
- **Monitor relay lag.** A stalled relay is silent: writes still succeed and nothing downstream happens
- **Store the event in its published shape**, not the raw row — consumers should not depend on your schema

CDC is the alternative relay. Instead of polling a table, read the database's replication log directly. Lower latency and no polling — at the cost of a pipeline component, and events shaped like rows rather than domain facts unless you still write to an outbox table and capture *that*.

Inbox — consuming exactly once, in effect

```
BEGIN
  // has this message already been handled?
  INSERT INTO inbox (message_id) VALUES (?)
    ON CONFLICT DO NOTHING          // duplicate → 0 rows
  IF inserted = 0 → COMMIT and return   // skip

  … apply the business effect …
COMMIT          // dedup record + effect, atomically
```

The record and the effect must share one transaction. Marking a message processed in a separate step reintroduces exactly the dual-write problem the outbox just solved, one layer down.

Delivery semantics, stated plainly

Semantic	Reality
At-most-once	Fire and forget. Messages can be lost. Acceptable only for genuinely disposable signals
At-least-once	What brokers actually give you. Duplicates **will** happen — on redelivery, rebalance, retry and replay
Exactly-once	Not achievable end to end across systems. **At-least-once delivery + idempotent processing** produces the same observable effect, and is what "exactly-once" means in practice

Size the inbox retention to the redelivery window. It must outlive the longest possible retry or replay — otherwise a message redelivered after cleanup is processed a second time, and the deduplication silently stops working.

Outbox and inbox are a pair. Outbox guarantees the event is published; inbox guarantees it is applied once. Deploying one without the other leaves an obvious gap at the other end.

8 · IDEMPOTENCY

what makes retries safe

An operation is idempotent when applying it twice has the same effect as applying it once. **Every retry, redelivery and replay on this sheet depends on it** — without it, reliability mechanisms become duplication mechanisms.

The fact that forces this. A timeout tells you **nothing** about whether the other side did the work. The request may have been received, processed and the response lost. Retrying without idempotency charges the customer twice.

How to achieve it

Technique	Use
Naturally idempotent	`set status = SHIPPED` is safe to repeat. Prefer this shape where the domain allows
Idempotency key	Caller supplies a unique key; the server stores it with the result and returns the same result on a repeat
Deduplication table	The inbox pattern — record the message id in the same transaction as the effect
Conditional write	`UPDATE … WHERE version = ?` — the second attempt affects zero rows
Business key	A unique constraint on a natural identifier lets the database reject the duplicate

Return the original result, not an error. A repeated request with the same idempotency key should return the first response. Returning a conflict makes the caller believe it failed, and they will retry again.

Ordering, which is separate

- **Order is per partition, never global.** Partition by aggregate id to keep one entity's events in sequence
- **Handle out-of-order arrival** — carry a version or sequence number and ignore anything older than current state
- **Idempotency does not give you ordering**, and ordering does not give you idempotency. Both are needed

Where the key comes from matters. Generate it at the point the intent is formed — the client, or the start of the saga — and carry it end to end. A key generated at the retry site is a new key every time, which defeats the whole mechanism.

9 · EVENT SOURCING

state as a consequence

Store **what happened** as an immutable, append-only sequence. Current state is derived by replaying it. The events are the source of truth; the state is a cache of them.

```
// instead of: balance = 150
AccountOpened      {id, owner}
Deposited          {amount: 200}
Withdrawn          {amount: 50}
// balance is computed: 0 + 200 − 50 = 150
// and you can also answer "what was it in March?"
```

What it uniquely gives you. A complete audit trail by construction, temporal queries ("what did we believe at the time?"), the ability to build a brand-new read model by replaying history, and debugging by reproducing the exact sequence that produced a bad state.

A correction worth noting: writes are *not* a weakness. Appending is among the cheapest operations a database performs, so event sourcing handles high write volume well. The real costs are elsewhere — in the three items below.

The three real costs

Replay time	A long stream is slow to rebuild. **Snapshots** — periodic materialised state — are not optional past a certain stream length
Schema evolution	Events are immutable and live for ever, so old versions must remain readable. Version every event and write **upcasters** that translate old shapes forward
Erasure	An immutable log conflicts directly with a right-to-erasure request. The usual answer is **crypto-shredding** — encrypt personal data per subject and destroy the key

Decide the erasure approach before the first event is written. Retrofitting it onto an existing log means rewriting history, which is precisely what the design was built to prevent.

Events describe facts, in the past tense — `OrderPlaced`, not `PlaceOrder`. A command can be rejected; an event already happened and cannot be.

10 · CQRS

two models, one truth

Separate the model you **write** through from the model you **read** through. Writes go to a normalised model optimised for invariants; reads come from projections shaped for the queries that actually run.

The correction that saves a lot of confusion: CQRS and event sourcing are independent. You can use CQRS over an ordinary relational database with a materialised view, and you can use event sourcing without CQRS. They combine well, which is why they are usually taught together — but neither requires the other, and adopting both at once is a large step.

Why it helps

- **The shapes genuinely conflict.** A model good for enforcing invariants is normalised; a model good for a dashboard is not
- **Read and write scale independently** — usually asymmetric by a large factor
- **Several read models** from one write model: a search index, a cache, a reporting table
- **Rebuildable** — a projection can be dropped and regenerated, which makes read-side changes low-risk

What it costs

- **Replication lag is user-visible.** A user writes, then immediately reads, and does not see their own change
- Two models to keep in step, and projection failures to detect
- More moving parts than most systems need

Handle read-your-writes explicitly. Read from the write model immediately after a write, return the new state in the write's own response, or show the pending state in the interface. Doing nothing means users see their change vanish and submit it again.

Apply it per aggregate, not per system. CQRS is worth it for the few areas with a genuine read/write mismatch. Applying it uniformly across every entity multiplies the machinery without the benefit.

11 · LOCKS, LEASES & CONCURRENCY

including one that does not work

Optimistic concurrency — start here

```
UPDATE inventory
   SET qty = qty − 1, version = version + 1
 WHERE id = ? AND version = ?
// 0 rows affected = someone else won. Re-read, retry.
```

The cheapest correctness tool available, and the one most often missing. No lock service, no coordination, no extra component — just a column and a retry. It solves lost updates on a single record completely.

Distributed locks — and the flaw

A lock with an expiry is not mutual exclusion. Client A acquires a 30-second lease, then stalls — a long garbage-collection pause, a scheduling delay, a slow network. The lease expires. Client B acquires it legitimately. Client A resumes, **believing it still holds the lock**, and writes. Two writers, no error, silent corruption.

```
// the fix: a fencing token
acquire → returns monotonically increasing token
write(data, token)
// the storage layer rejects any token lower
// than the highest it has already seen

A holds token 33 → stalls
B acquires token 34 → writes ✓
A resumes, writes with 33 → rejected ✓
```

Fencing requires support from the resource being protected — it must check and remember the token. Without that, a distributed lock is a best-effort optimisation to reduce contention, and must not be relied on for correctness.

Choosing

Single record	Optimistic concurrency — version column
Short critical section, one database	A database transaction, or `SELECT … FOR UPDATE`
Single-runner jobs	A lease is fine — duplicate execution is wasteful, not incorrect
Correctness-critical exclusion	Fencing tokens, or redesign so the invariant lives in one service and one transaction

12 · CONSISTENCY MODELS

what "consistent" means

Model	Guarantee
Linearizable strong	Every read sees the latest committed write, as if there were one copy. Requires coordination on every operation
Sequential	Everyone observes operations in the same order, though not necessarily in real time
Causal	Operations that are causally related are seen in order; unrelated ones may differ per observer
Read-your-writes	You always see your own updates. Often the minimum users actually notice
Monotonic reads	Time never appears to move backwards for one reader
Bounded staleness	Reads lag by at most a stated time or version count — the useful middle ground
Eventual	Replicas converge if writes stop. The weakest guarantee that is still useful

"Eventually consistent" needs a number. Eventually is 50 milliseconds in one system and 20 minutes in another under load. State the target convergence window, measure it at p99, and alert when it is exceeded — otherwise it is not a design decision, it is a hope.

Getting the guarantee you need

- **Quorums** — with N replicas, reading R and writing W, `R + W > N` guarantees a read sees the latest write
- **Read from the leader** for operations that must be current
- **Sticky routing** to one replica gives read-your-writes and monotonic reads cheaply
- **Version tokens** — the client passes the version it last saw, and the read waits for a replica at least that current

Choose per operation, not per system. "Show my order after I place it" needs read-your-writes; "show the dashboard" tolerates minutes. Applying one model everywhere means paying the strongest cost for the weakest need.

The user interface is part of the design. Showing a pending state honestly — "payment processing" — turns an inconsistency window into normal behaviour rather than a bug report.

13 · CHOOSING, AND A WORKED EXAMPLE

the decision path, and what an e-commerce order actually looks like when all of it is applied

The decision path

Situation	Pattern
Data changes together and is owned by one service	**One local transaction.** Stop here — this is the correct answer far more often than it is used
Two services always change together	Question the boundary before adding a pattern
Concurrent updates to one record	**Optimistic concurrency** — version column
A service must tell others something happened	**Outbox + idempotent consumer**
Multi-step process with a natural undo	**Saga** — orchestrated past three steps
A resource must be held before committing	**TCC**
Reads and writes have conflicting shapes	**CQRS**, on that aggregate only
History and audit are the requirement	**Event sourcing**
Events needed from a system you cannot change	**CDC**
Genuinely need atomic commit, small trusted boundary	**2PC** — and confirm it is really needed

A worked order flow

```
1 Order service — one local transaction
    INSERT order (status = PENDING)   ← semantic lock
    INSERT outbox (OrderPlaced)
    COMMIT                            ← atomic ✓

2 Relay publishes OrderPlaced   at-least-once

3 Payment service — inbox dedup on message id
    charge with the saga's idempotency key
    INSERT outbox (PaymentCaptured)   same tx ✓

4 Inventory — reserve, not deduct   ← TCC-style hold
    reservation expires in 15 min if unconfirmed

5 Order service — status = CONFIRMED  ← pivot step

6 Shipping, email                     ← irreversible, last

FAILURE at 4: compensate in reverse —
    refund payment, release nothing, order = CANCELLED
    compensation fails → retry → DLQ → human + alert
```

Read the shape, not the steps. Every reversible action happens before the pivot; everything irreversible happens after it. The semantic lock makes the in-flight state explicit to other services and to the customer. Nothing anywhere depends on a distributed transaction.

What the customer sees matters as much as the mechanism. Between steps 1 and 5 the order genuinely is pending — so show it as pending. The consistency window becomes a status rather than a defect.

14 · READINESS CHECKLIST

confirm before a distributed process handles real money or real inventory

- ☐ **Service boundaries checked** — nothing that always changes together was split apart
- ☐ **Data owned by one service** changes in one local transaction
- ☐ **Optimistic concurrency** on every mutable record
- ☐ **No dual write anywhere** — no code writes the database and publishes separately
- ☐ **Outbox written in the same transaction** as the state change
- ☐ **Outbox pruning scheduled** and the relay's lag monitored
- ☐ **Every consumer idempotent**, with the dedup record in the same transaction as the effect
- ☐ **Inbox retention exceeds** the longest redelivery or replay window
- ☐ **Idempotency keys generated at intent** and carried end to end
- ☐ **Repeated requests return the original result**, not a conflict
- ☐ **Ordering requirements identified**, and partitioning matches them
- ☐ **Out-of-order arrival handled** by version or sequence number
- ☐ **Every saga step has a compensating action**, defined before it shipped
- ☐ **Compensations are idempotent** and safe to retry
- ☐ **Compensation failure has a defined path** — retry, DLQ, alert, runbook, human
- ☐ **Irreversible steps ordered last**, after the pivot
- ☐ **Saga state is durable** and resumes after a coordinator crash
- ☐ **Timeout on every step**, with a defined action on expiry
- ☐ **Isolation anomalies considered** — dirty read, lost update, fuzzy read
- ☐ **Semantic locks** on records that are in-flight
- ☐ **Reservations expire**, and expiry rate is monitored
- ☐ **Correlation id** threaded through every message and log line
- ☐ **Convergence window stated as a number** and measured at p99
- ☐ **Read-your-writes handled** where users write then immediately read
- ☐ **Pending state shown honestly** in the interface
- ☐ **Distributed locks not relied on for correctness** without fencing tokens
- ☐ **Event schema versioned**, with upcasters for old shapes
- ☐ **Snapshots** where event streams grow long
- ☐ **Erasure approach decided** before the first event is written
- ☐ **Stuck-saga alert** — anything in an intermediate state beyond its expected window
- ☐ **DLQ monitored**, with an owner and a documented drain procedure
- ☐ **Failure paths tested** by deliberately failing a middle step under load

The one that summarises the sheet: **take one business process, fail step three, and follow what happens to the data, the customer and the operator.** If the answer is "it would probably retry", or nobody can say where the order ends up, the design is not finished — and this is exactly the case that will occur, because at scale every intermediate state eventually happens.